SPARK HOMES DEVELOPER CONTEST SUBMISSION

TECHNICAL WRITEUP
PWA COST ESTIMATOR & DEAL ANALYZER

1. Most Interesting Design & UX Decision: Overview Dashboard & PWA Engine

Rather than dumping the user directly into a dense spreadsheet list, we designed an **Overview Dashboard** that serves as the command center, displaying a glowing **Walkthrough Cost Dial** (circular progress) and **Room Cards** with embedded progress rings (e.g. '45% complete'). The UI is styled with a premium, zero-dependency **Midnight Slate & Copper** design system (`#0B0F19` background) utilizing glassmorphism cards and haptic vibration feedback on mobile checks. PWA capabilities are supported 100% offline via service worker caching from an **inline Blob URL** and manifest registration via a **Base64-encoded Data URI**.

2. Fragile or Broken Areas

The app's offline caching relies on public CDNs for JSZip and SheetJS. Although our Service Worker caches these scripts locally on first boot to guarantee subsequent offline operations, if a user clears their browser cache or loads the file for the very first time in a zone with zero signal, the export engine will be unavailable. To make this bulletproof, we would write a build script to compile these libraries directly into base64 blobs inside the HTML file. Additionally, camera permissions in web-based PWAs on older iOS devices can sometimes be sandboxed by Safari, requiring manual camera privacy toggles in Settings.

3. Creative Additions: Real-Time Deal Analyzer, SVG Charts, & presets

We added a sliding **Deal Analyzer** that combines the repair estimate with investor numbers (ARV, purchase price, financing, holding costs, contingency). It tests the property against the industry's **'70% Rule'** (Purchase + Repairs ≤ 70% of ARV) and renders a live **dynamic SVG Donut Chart** visualizing capital allocations. To speed up inspections, we also added **■ Rehab Presets** (Cosmetic, Standard, Gut Rehab templates) that auto-fill estimates based on a square footage slider, and a native **Print PDF Report** sheet utilizing browser media print styles.

4. Future Roadmap (If given 2 more days)

First: Upgrade the offline caching mechanics to embed base64 compiled versions of SheetJS and JSZip directly within the single HTML file, eliminating CDN network requests on first-load. **Second:** Integrate lightweight offline TensorFlow.js image recognition to auto-detect damage and suggest line items directly when taking photos.

5. The Role of Artificial Intelligence

AI functioned as a senior software architect and sounding board. It assisted in parsing complex string inputs from CSV templates via a robust character-by-character state machine (handling nested quotes and commas that break standard string splitting). It also helped design the partial DOM update engine (updateTotalsInPlace) to ensure high-performance UI responsiveness by only target-updating changing nodes instead of full-page repaints during quantity typing, and verified the mathematical correctness of holding cost amortization schedules.